

Mini-projet 05 — Gestion d'erreurs fine

Module A · dernier · Notions : **hiérarchie d'exceptions** · **try/except/else/finally** · **raise** · **exceptions maison** · Durée : 1 h 30 – 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Un outil qui plante à la première ligne bizarre est inutilisable. Un bon outil **distingue** les problèmes (« cette cible est mal écrite » ≠ « le réseau est coupé »), **continue** malgré une erreur isolée, et donne des **messages clairs**. C'est ce qui sépare un script du dimanche d'un outil pro. Et ça reprend ton point du MP01 (test n°7) où distinguer deux erreurs t'avait posé souci — on le règle ici.

1. Rappel : try / except

```
try:
    port = int("abc")          # ça va lever une erreur
except ValueError:
    print("pas un nombre !")  # on rattrape SEULEMENT ValueError
```

Règle d'or : attrape l'**exception précise** que tu attends (`ValueError` , `KeyError` ...), **jamais** un `except:` nu ni `except Exception` « au cas où ». Sinon tu masques des bugs que tu voudrais voir.

2. La hiérarchie des exceptions

Toutes les exceptions Python héritent d'une classe de base, `Exception`. C'est un arbre :

```
Exception
├─ ValueError  (mauvaise valeur : int("abc"))
├─ TypeError   ("a" + 1)
├─ KeyError    (dict["clé absente"])
├─ IndexError  (liste[99])
├─ OSError
│   ├─ ConnectionRefusedError (port fermé)
│   └─ TimeoutError          (pas de réponse)
└─ ... (et tes exceptions à toi, plus bas)
```

Conséquence pratique : attraper une classe **parente** attrape aussi ses enfants. `except OSError` rattrape à la fois `ConnectionRefusedError` ET `TimeoutError`. Mais si tu veux les **distinguer**, tu les mets séparément, **du plus précis au plus général** :

```
try:
    connexion()
except ConnectionRefusedError:
    print("port fermé")      # cas précis 1
except TimeoutError:
    print("injoignable")     # cas précis 2
except OSError:
    print("autre souci réseau") # filet de sécurité général, EN DERNIER
```

L'ordre compte : Python teste les `except` de haut en bas et s'arrête au premier qui matche. Si tu mets `except OSError` **en premier**, il avalera tout et tes cas précis ne seront jamais atteints.

Attraper plusieurs erreurs d'un coup

```
except (ValueError, KeyError):  # un tuple = "l'une OU l'autre"
    print("donnée invalide")
```

3. else et finally

```
try:
    f = open("data.txt")
except FileNotFoundError:
```

```

    print("fichier absent")
else:
    print("ouvert !")    # else = "si AUCUNE erreur n'a eu lieu"
finally:
    print("terminé")    # finally = "TOUJOURS, erreur ou pas" (nettoyage)

```

Bloc	Quand il s'exécute
<code>try</code>	le code risqué
<code>except</code>	seulement si l'erreur correspondante survient
<code>else</code>	seulement si aucune erreur n'est survenue
<code>finally</code>	toujours (succès ou échec) — idéal pour fermer/nettoyer

4. Lever une erreur : raise

`raise` déclenche volontairement une exception. Utile quand TA fonction détecte un problème :

```

def valider_port(port):
    if port < 0 or port > 65535:
        raise ValueError(f"port hors bornes : {port}")
    return port

```

5. Tes propres exceptions notion clé

Tu peux créer une exception « maison » : c'est une simple classe qui hérite de `Exception` . Ça rend ton code lisible et permet à l'appelant d'attraper **précisément ton erreur à toi**.

```

class CibleInvalide(Exception):
    """Levée quand une ligne de cible est mal formée."""
    pass

# On la lève comme n'importe quelle autre :
raise CibleInvalide("pas de ':' dans la ligne")

# Et on l'attrape par son nom :
try:
    parser_cible("nawak")
except CibleInvalide as e:
    print(f"cible rejetée : {e}")    # e contient ton message

```

Le `as e` : il capture l'objet exception dans la variable `e` . `str(e)` (ou `f"{e}"`) te redonne le message que tu as mis dans le `raise` .

Ta mission

Objectif : un parseur de cibles **robuste** qui rejette proprement les lignes invalides sans jamais planter.

- Crée une exception maison `class CibleInvalide(Exception)` .
- Écris `parser_cible(ligne)` qui renvoie `{"IP": ip, "PORT": port_int}` , MAIS lève `CibleInvalide` avec un **message clair** si :
 - la ligne n'a pas exactement un `:` (→ message « séparateur manquant ») ;
 - le port n'est pas un nombre (→ message « port invalide : ... »).
- Écris `parser_toutes(lignes)` qui parcourt une liste de lignes, et pour chacune :
 - si elle est valide → l'ajoute à la liste résultat ;
 - si `CibleInvalide` est levée → affiche l'erreur et **continue** (les autres lignes doivent quand même être traitées).

Renvoie la liste des cibles valides.

Test :

```

lignes = ["192.168.1.10:22", "machin", "10.0.0.5:abc", "10.0.0.5:443"]
valides = parser_toutes(lignes)

# Doit afficher 2 erreurs (pour "machin" et "10.0.0.5:abc")
# et valides doit contenir 2 cibles : 192.168.1.10:22 et 10.0.0.5:443
print(valides)

```

```
# -> [{'IP': '192.168.1.10', 'PORT': 22}, {'IP': '10.0.0.5', 'PORT': 443}]
```

Bonus : ajoute un bloc `finally` dans `parser_toutes` qui affiche à la fin « X cibles valides, Y rejetées ».

✓ Critères de réussite

- `CibleInvalide` hérite bien de `Exception` .
- Une ligne invalide **n'arrête pas** le traitement des suivantes.
- Chaque erreur affiche un message compréhensible (pas juste « erreur »).
- Tu attrapes `CibleInvalide` précisément (pas un `except: nu`).

Si tu bloques — indices progressifs

Étape	Indice
Détecter le mauvais séparateur	Après <code>ligne.split(":")</code> , regarde la longueur de la liste obtenue : si elle ne fait pas 2, c'est invalide → <code>raise CibleInvalide(...)</code> .
Détecter le port non-numérique	Entoure le <code>int(port)</code> d'un <code>try/except ValueError</code> , et dans le <code>except</code> , <code>raise CibleInvalide(...)</code> avec un message parlant.
Continuer malgré une erreur	Dans <code>parser_toutes</code> , mets le <code>try/except CibleInvalide</code> à l'intérieur de la boucle <code>for</code> , autour de l'appel à <code>parser_cible</code> . Comme ça une erreur ne casse qu'un tour de boucle.
Compter (bonus)	Deux compteurs que tu incrémente, affichés dans le <code>finally</code> .

Où ranger : `mini-projets/05-gestion-erreurs/parseur_robuste.py` .

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ». Ce sera le **dernier du Module A** — on fera un petit bilan avant d'attaquer la POO.